

First App Notes

Setting up Python, **uv**, a virtual environment and calling the OpenAI API from your very first script.

INSTRUCTOR Abbas	PLATFORM Telusko.com	ENVIRONMENT Windows + VS Code	LEVEL Beginner
----------------------------	--------------------------------	---	--------------------------

What you will build

A small Python program that sends a question to the OpenAI Chat Completions API and prints the reply in your terminal. The API key is kept safely outside the code, inside a **.env** file.

Steps at a glance

- | | |
|-------------------------------|---------------------------------|
| 1. Check Python | 6. Create the .env file |
| 2. Install uv | 7. Project structure |
| 3. Add uv to PATH | 8. Write main.py |
| 4. Verify uv | 9. Run the app |
| 5. Set up the project with uv | 10. Second example (OpenAI SDK) |

STEP 1 | Check that Python is installed

Open a terminal (PowerShell) and run:

TERMINAL

```
python --version
```

If a version number is printed, Python is installed and you can continue. If you get an error, install Python first.

STEP 2 | Install uv

uv is a fast package and project manager for Python. Install it for your user account:

TERMINAL

```
python -m pip install --user uv
```

STEP 3 | Add uv to the PATH

Option A — for the current terminal only

Quick and temporary. It works until you close this terminal window.

POWERSHELL

```
$env:Path += ";$env:APPDATA\Python\Python314\Scripts"
```

Option B — permanent (recommended)

Set it once through Windows so every new terminal knows where uv is:

- Windows → Search → **Environment Variables**
- Open **Edit the system environment variables**
- Click **Environment Variables**
- Under **User variables**, select **Path** → **Edit** → **New**
- Paste the folder path shown below and confirm with OK on every window

PATH TO ADD

```
C:\Users\haide\AppData\Roaming\Python\Python314\Scripts
```

Note: Replace **haide** with your own Windows username, and **Python314** with your installed Python version if it is different.

STEP 4 | Verify the installation

Close the terminal, open a new one, and check:

TERMINAL

```
uv --version
```

A version number here means uv is installed correctly and is on your PATH.

STEP 5 | Create the project

Create an empty folder and open it in VS Code, then open the built-in terminal and run these commands **one by one**:

TERMINAL

```
uv init
uv venv
.venv\Scripts\activate
uv add requests python-dotenv
```

Command	What it does
<code>uv init</code>	Creates the project files, including <code>pyproject.toml</code>
<code>uv venv</code>	Creates the <code>.venv</code> virtual environment folder
<code>.venv\Scripts\activate</code>	Activates the environment for this terminal
<code>uv add requests python-dotenv</code>	Installs the two libraries the app needs

Note: After activation your terminal prompt shows **(.venv)** at the start. If it does not, the environment is not active.

STEP 6 | Create the .env file

In the project folder create a file named exactly:

FILE NAME

`.env`

Add this single line inside it:

`.ENV`

`OPENAI_API_KEY=YOUR_NEW_API_KEY`

Important: Never paste your API key directly into `main.py` and never share or commit the `.env` file. If a key is ever exposed, revoke it and generate a new one.

STEP 7 | Project structure

Once everything is created, your folder should look like this:

```
code/
|
|-- .venv/
|-- .env
|-- main.py
|-- example2.py
|-- pyproject.toml
|-- README.md
```

STEP 8 | Write main.py

Open **main.py** and type the following code:

MAIN.PY

```
import os
import requests
from dotenv import load_dotenv

# Load variables from .env
load_dotenv()

# Read OpenAI API key
api_key = os.getenv("OPENAI_API_KEY")

# OpenAI Chat Completions API endpoint
uri = "https://api.openai.com/v1/chat/completions"

# Send API key in Authorization header
headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {api_key}"
}

# Request sent to the model
payload = {
    "model": "gpt-4o-mini",
    "messages": [
        {
            "role": "user",
            "content": "What is the capital of France?"
        }
    ]
}

# Call OpenAI API
response = requests.post(
    uri,
    headers=headers,
    json=payload
)

# Print API response
print(response.json())
```

STEP 9 | Run the app

With the virtual environment still active, run:

TERMINAL

```
python main.py
```

The terminal prints the full JSON response from OpenAI. The answer itself sits inside `choices → 0 → message → content`.

STEP 10 | Second example — using the OpenAI SDK

The first example called the API manually with **requests**. The official **openai** library does the same job with much less code. Install it first:

TERMINAL

```
uv add openai
```

Now create a **new file** next to `main.py`:

FILE NAME

```
example2.py
```

EXAMPLE2.PY

```
from openai import OpenAI
from dotenv import load_dotenv

# Load variables from .env
load_dotenv()

client = OpenAI()

response = client.responses.create(
    model="gpt-4o-mini",
    input="tell me about ai in short.",
)

print(response.output_text)
```

Run it:

TERMINAL

```
python example2.py
```

Note: `OpenAI()` picks up the key from the environment on its own, so there is no headers or Authorization code to write. `load_dotenv()` is what puts `OPENAI_API_KEY` from your `.env` into the environment, so keep that line.

	main.py (requests)	example2.py (SDK)
Library	requests	openai
URL / headers	Written by hand	Handled for you
API key	Passed in the Authorization header	Read automatically from the environment
Reading the reply	<code>response.json()</code> — full JSON	<code>response.output_text</code> — just the answer

Quick reference — all commands in order

CHEAT SHEET

```
python --version
python -m pip install --user uv
$env:Path += ";$env:APPDATA\Python\Python314\Scripts"
uv --version
uv init
uv venv
.venv\Scripts\activate
uv add requests python-dotenv
python main.py
uv add openai
python example2.py
```

If something goes wrong

Problem	Fix
uv is not recognised	PATH is not set. Redo Step 3 and open a brand new terminal.
<code>ModuleNotFoundError</code>	The virtual environment is not active, or the library was not added. Activate <code>.venv</code> and run <code>uv add</code> again.
Response says invalid API key	Check the <code>.env</code> spelling, make sure there are no quotes or spaces around the key, and confirm the file sits next to <code>main.py</code> .
Activate script blocked	PowerShell execution policy is blocking it. Use the terminal inside VS Code, or allow scripts for your user.